



THE PROBLEM

As GPUs in the market become more powerful, equipped with increasing number of cores and memory bandwidth, it has become more attractively feasible to parallelize computationally expensive applications native to the GPU that exceed the performance of the CPU.

Seam carving and content aware image retargeting techniques, originally developed by Ariel Shamir and Shai Avidan of MERL **is an application that allows intelligent image resizing without the distortion of scaling or the data loss of cropping**. This operation can become quite slow due to heavy arithmetic operations, backtracking and array operations as image resolution grows.

THE SOLUTION

By using NVIDIA's CUDA, a development toolkit enabling the parallel computing on NVIDIA GPUs, we have developed a parallelized version of Shamir and Avidan's algorithm. Our project is called **seamster**, an application that fully takes advantage of the large amount of cores modern GPUs have to offer.

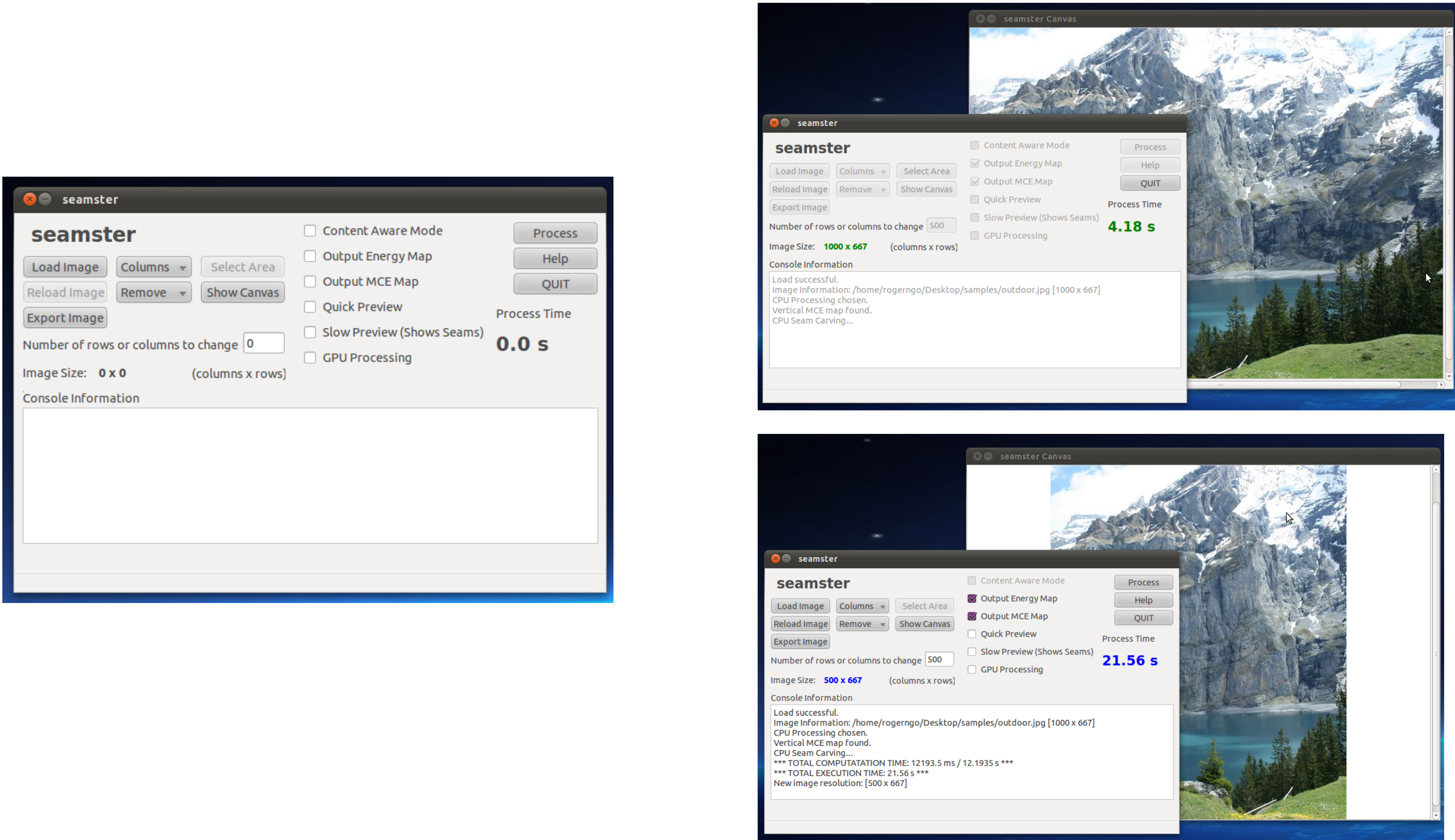
APPLICATIONS FOR IMAGE RETARGETING

Image retargeting techniques can be useful when we are interested in resizing image content without loss of data (cropping) or distortion (scaling). Image retargeting allows images to be resized by deleting or adding parts of the image which are least noticeable by the human eye. Imagine resizing your *favorite* wallpaper that fit perfectly for your 5:4 monitor to your new awesome 16:9 screen without distorting it!

APPLICATION FEATURES

- * Easy to use graphical user interface
- * JPG, PNG, TIFF and BMP image support
- * Fully parallelized seam carving, insertion and content aware delete for CUDA GPUs
- * Seams and resized image can be shown during carving
- * NVIDIA CUDA capable card not required to run the program

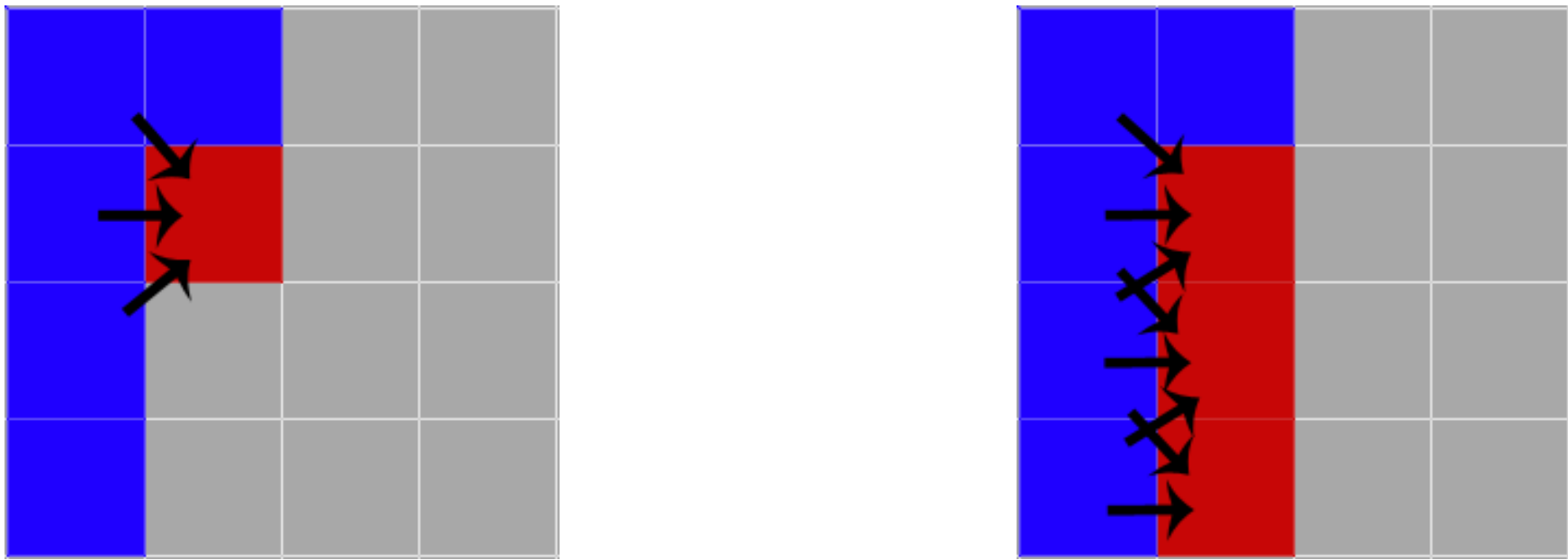
SEAMSTER SCREENSHOTS



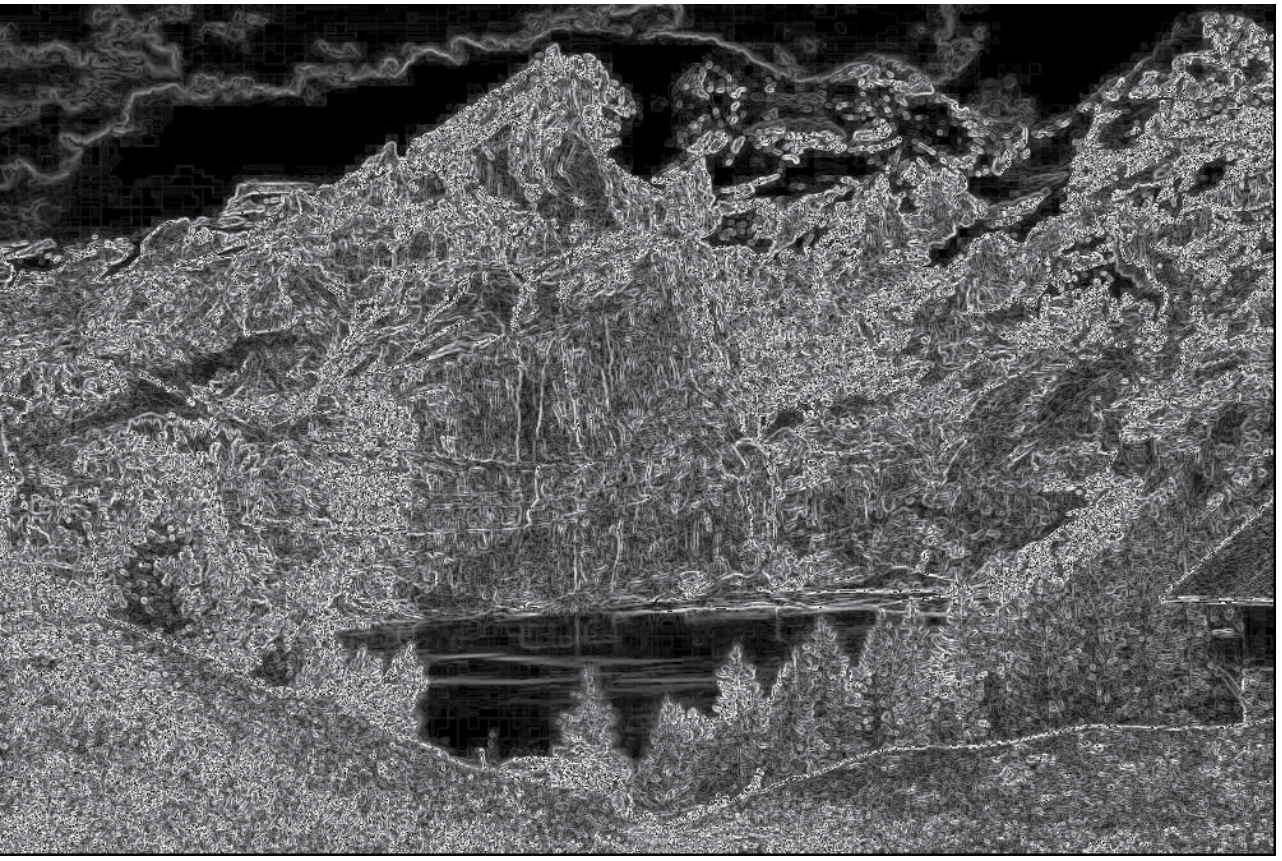
BRIEF OVERVIEW OF PARALLEL SEAM CARVING

1. Calculate the energy map. We used the Sobel operator to apply convolution to every pixel in the image. Parallelizing this step involved assigning CUDA threads to invidual pixels in the image. The resulting image is a map that shows the densities and edges of certain areas in the image.
2. When the energy map has been acquired, the Minimum Cumulative Energy map is found by applying dynamic programming to the energy map. We use the first row/column of the energy map as base cases. In finding the value, we take the minimum value of three neighboring candidates.

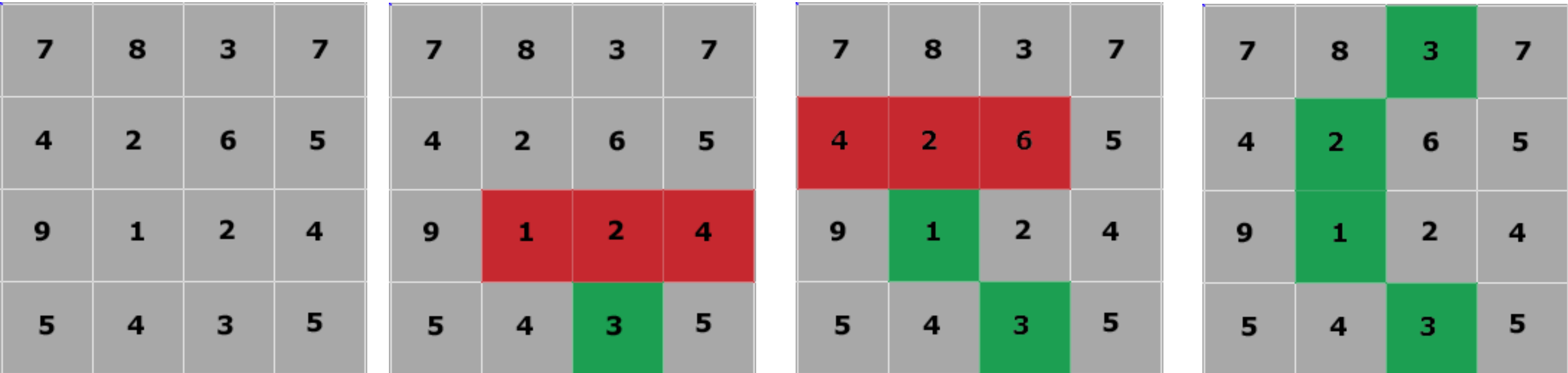
Left: CPU MCE
Right: GPU MCE



The energy map of an image



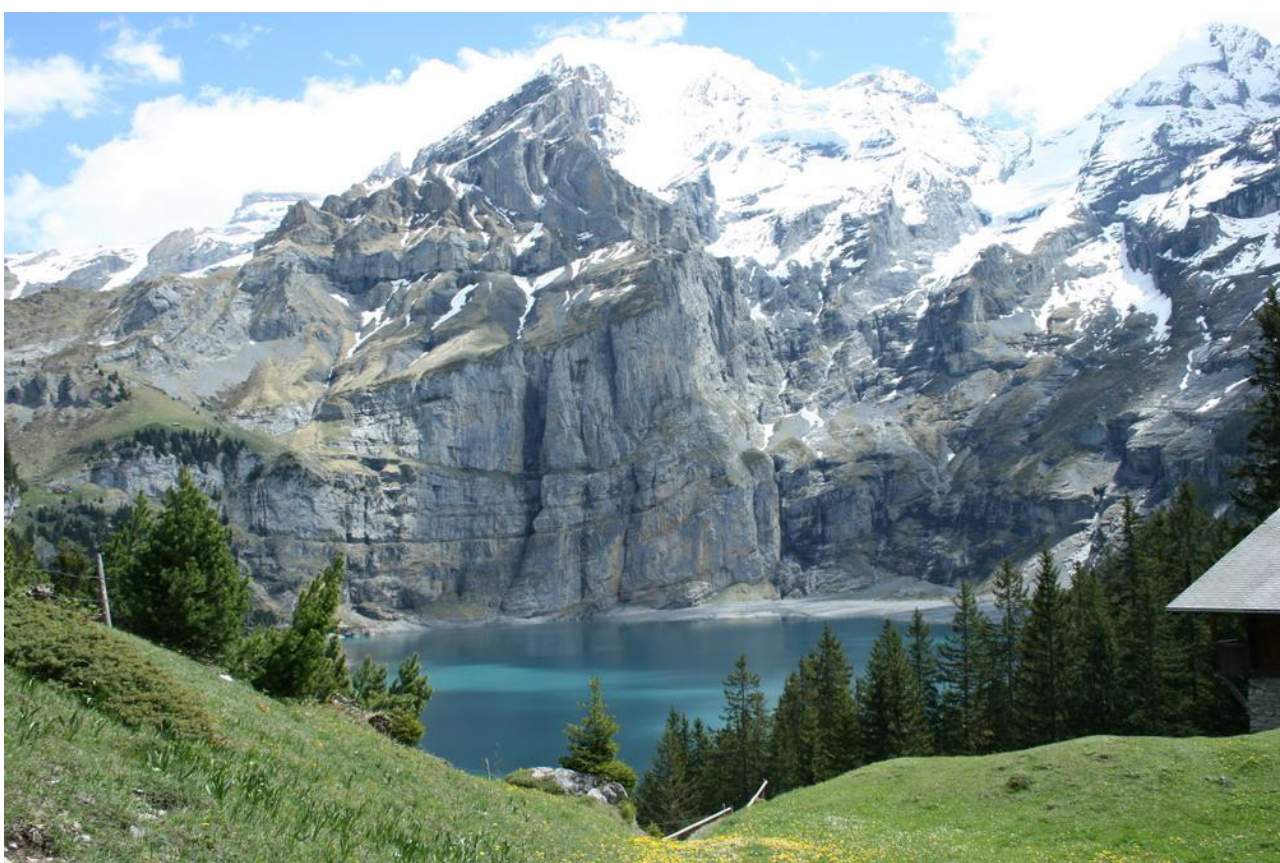
3. Backtracking on the MCE is then used to find the optimal seam. Below shows an example of finding the optimal vertical seam.



The minimum value of the last row of the image is found. Then three candidate pixels: the immediate left, top and right are chosen to determine which will be included in seam path. The minimum of these three is chosen and the process repeats until a path from the bottom of the image to the top has been formed.

A shift is then performed to overwrite the seam in the map.

To parallelize this step, we first implemented a way to find all the seams of an image at once. By finding all the seams at once, some are not guaranteed to be optimal. In parallel shifting, if the pixel is after the seam path, then a check is performed whether or not the current value is part of another seam. If the value is, then we retrieve its pixel value N and jump to the next N non-seam pixels. This operation is repeated for all pixel values that are part of the seam. All non-seam pixels are then stored in a new image.



Original Image (1000x667) JPEG



CPU Result (500x667)



GPU Result (500x667)

PERFORMANCE RESULTS

When tested on a 1000 x 667 JPEG image, we observed about a 46x performance increase by parallelizing seam carving on the GPU.

Calculations and Operations	CPU (Serial)	GPU (Parallel)
Energy Map (Computation Only)	0.095 s	0.001 s
MCE Map (Computation Only)	0.116 s	0.030 s
Seam Carving (Computation Only)	6.306 s	0.184 s
Other Operations (Allocation, transfer, traversal, etc...)	4.103 s	0.015 s
Total time	10.620 s	0.230 s

